

KEYSTONE BANK LIMITED

FIRS e-Invoice Agent

Technical Architecture & Component Reference

FIRS Compliance | Nigeria Revenue Service Integration

Document Date: 22 June 2026

Environment: **PREPROD** (Cryptware)

Purpose: This document describes the architecture of the Keystone Bank FIRS e-Invoice compliance agent — a Python-based middleware that automatically reads transaction records from the core banking SQL Server database, converts them to FIRS-compliant invoice payloads, submits them to the Cryptware PREPROD API (Nigeria Revenue Service portal), and stores the returned Invoice Reference Numbers (IRN) and QR codes back in the database. A Flask web dashboard provides real-time visibility into submission status, error tracking, and printable invoices.

1. Database Connection

The agent connects to a Microsoft SQL Server instance named **KBL-FIRS-SVR** using the **pymssql** driver (a pure-Python TDS implementation that works without native ODBC drivers). All database credentials are stored in **config.json** at the project root and loaded at startup. Both the scheduler and the Flask dashboard share the same connection parameters.

Connection parameters (config.json → database section):

Server	KBL-FIRS-SVR (SQL Server hostname on the Keystone Bank internal network)
Database	FIRSINVOICEDB
User	FIRSUser
Password	Stored in config.json (not committed to version control via .gitignore)
Timeout	30 seconds per query
Driver	pymssql — installed inside the project venv; no system-level ODBC required

Source table — FIRSINVOICEDB.dbo.REPORT:

The core banking ERP writes one row per transaction fee charge into the REPORT table. The agent reads from this table and writes back four fields after a successful API call.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Type' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Type', textColor=Color(.419608,.447059,.501961,1), us_lines=[])]) 'style': 'bulletText': None 'debug': 0) #Paragraph		
TRANS_REF	nvarchar	Unique invoice identifier (document_identifier in API)
BOOKING_DATE	datetime	Invoice issue date
INVOICE_TYPE	smallint	381 = Sales Invoice, 380 = Credit Note, 384 = Debit Note
AMOUNT_LCY	decimal	Invoice amount in local currency (NGN)

CUSTOMER_SHORT_NAME	nvarchar	Customer display name
ACCOUNT_NO	bigint	Customer bank account number
TIN	nvarchar	Tax Identification Number (empty → B2C transaction)
PHONE_1	nvarchar	Customer phone (normalized to E.164 format)
STREET / TOWN_COUNTRY / COUNTRY	nvarchar	Postal address fields
POST_CODE	nvarchar	Postal zone (falls back to 100001 if absent)
SERVICE_FEE_NAME	nvarchar	Invoice line description (e.g. MOBILE BANKING INCOME)
HSN_CODE	nvarchar	Harmonized System / product code (e.g. 6499)
TAX_CATEGORY	nvarchar	STANDARD_VAT = 7.5% VAT; other = 0%
TRANS_REF3	nvarchar	Original IRN for credit notes (cancel_references)
REPORTING_STATUS	bit/int	NULL = pending, 1 = submitted OK, 0 = failed
IRN	nvarchar	Invoice Reference Number returned by Cryptware API
QR_CODE	nvarchar(MAX)	URL or base64 QR code from Cryptware API
ENVIRONMENT	varchar(10)	PREPROD or PROD — stamps which API handled the record
VALIDATION_ERROR	nvarchar	API error message stored on failure
LAST_UPDATED	datetime	Timestamp of last scheduler write
PREPROD_IRN	varchar(100)	Backup copy of IRN from preprod run (archived)
PREPROD_QR_CODE	nvarchar(MAX)	Backup copy of QR code from preprod run (archived)

Database setup (setup_database.sql):

```
ALTER TABLE [dbo].[REPORT] ADD
REPORTING_STATUS INT NULL,
IRN VARCHAR(100) NULL,
QR_CODE VARCHAR(MAX) NULL,
VALIDATION_ERROR VARCHAR(500) NULL,
LAST_UPDATED DATETIME NULL,
ENVIRONMENT VARCHAR(10) NULL,
PREPROD_IRN VARCHAR(100) NULL,
PREPROD_QR_CODE VARCHAR(MAX) NULL;
```

Important: The ERP writes the literal string "NULL" into nvarchar columns when a value is absent (rather than SQL NULL). The `_clean()` helper in `invoice_scheduler_pymssql.py` treats both SQL NULL and the string "NULL" as empty before building the API payload.

2. Cryptware API Configuration (PREPROD)

Keystone Bank submits FIRS-compliant invoices to the Nigeria Revenue Service (NRS) through **Cryptware Systems Ltd**, the licensed FIRS e-Invoice service provider. The integration uses the Cryptware PREPROD environment for testing and validation before production go-live.

PREPROD base URL	https://preprod-api.cryptwaresystemsLtd.com
Endpoint	POST /invoice/generate
Authentication	HTTP header: X-API-Key (no participant-id required)
API Key (preprod)	sk_live_f0ee420acf1ec33ec94150ab8534efc96e753cd6c5c8284ab511a14da64a0699
Success response	HTTP 201 Created (not 200)
Content type	application/json (request and response)
Request timeout	30 seconds

config.json structure:

```
{
  "database": {
    "server": "KBL-FIRS-SVR",
    "name": "FIRSINVOICEDB",
    "user": "FIRSUser",
    "password": "...",
  },
  "api": {
    "base_url": "https://preprod-api.cryptwaresystemsLtd.com",
    "api_key": "sk_live_f0ee420..."
  },
  "scheduler": {
    "interval_minutes": 5
  }
}
```

HTTP request headers sent with every submission:

```
Content-Type: application/json
X-API-Key: sk_live_f0ee420acf1ec33ec94150ab8534efc96e753cd6c5c8284ab511a14da64a0699
```

Environment detection: The scheduler automatically stamps each processed record with "PREPROD" or "PROD" in the ENVIRONMENT column by inspecting whether the word "preprod" appears in the configured base_url. This allows the database to track which records were submitted to preprod versus production, and supports the reset_for_production.sql workflow that clears preprod IRNs before the live production run.

3. Payload Transformation

The `map_to_api_payload()` method in `InvoiceScheduler` (`invoice_scheduler_pymssql.py`) converts raw database rows into the JSON structure required by the Cryptware `/invoice/generate` endpoint. The transformation handles data-quality issues common in bank ERP exports: literal "NULL" strings, invalid phone formats, non-ISO country codes, and short postal codes.

Invoice type mapping:

INVOICE_TYPE = 381	invoice_type_code: "381" — Commercial (Sales) Invoice
INVOICE_TYPE = 380	invoice_type_code: "380" — Credit Note (requires cancel_references)
INVOICE_TYPE = 384	invoice_type_code: "384" — Debit Note
Any other / NULL	Defaults to "381" — Sales Invoice

B2B vs B2C determination:

The `transaction_category` field is set dynamically based on whether a real TIN is present in the database. Bank retail customers typically have no TIN — they are classified B2C and a placeholder TIN (000000000000) is used to satisfy the API schema requirement.

```
tin = _clean(header["TIN"]) or "000000000000"
transaction_category = "B2B" if _clean(header["TIN"]) else "B2C"
```

Data normalisation helpers:

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Helper' 'frags': [ParaFrag (__tag__='para', bold=1, f ontName='Helvetica-Bold ', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Helper', textColor= Color(.419608,.447059,.5 01961,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Problem solved' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Problem solved', textColor =Color(.419608,.447059,.501961,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Logic' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Logic', textColor=Color(.419608,.447059,.501961,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>		
<code>_clean(val)</code>	ERP stores "NULL" string	Returns "" for None or string "NULL"; strips whitespace
<code>_format_phone(raw)</code>	E.164 format required by FIRS	Handles missing (→ +23400000000000), 0-prefix (→ +234...), 234-prefix (→ +234...), already +
<code>_normalise_country(raw)</code>	ERP stores dial codes ("234") or state strings ("08-ABUJA(FCT)")	Returns "NG" for anything that is not a 2-letter ISO alpha code
<code>_normalise_postal_zone()</code>	Postal codes like "1" or "234" too short	Returns "100001" (Lagos area fallback) for any value shorter than 5 characters

Full Cryptware payload structure (sales invoice example):

```
{
  "document_identifier": "FT26008025006184",
  "invoice_type": "STANDARD",
  "issue_date": "2026-01-08",
  "due_date": "2026-01-08",
  "invoice_type_code": "381",
  "document_currency_code": "NGN",
  "transaction_category": "B2C",
  "accounting_customer_party": {
    "party_name": "SAMUEL OBEDIENT O",
    "email": "noreply@na.ng",
    "telephone": "+2348036817304",
    "tin": "000000000000",
    "business_description": "SAMUEL OBEDIENT O",
    "postal_address": {
      "street_name": "UMUDINKWA AVODIM UBAKALA",
      "city_name": "UMUAHIA",
      "postal_zone": "100001",
      "country": "NG"
    }
  },
  "invoice_lines": [{
    "internal_id": "2043439688",
    "description": "MOBILE BANKING INCOME",
    "invoiced_quantity": 1,
    "price_amount": 7.97,
    "hsn_code": "6499",
    "price_unit": "PC",
    "product_category": "General",
    "tax_rate": 7.5,
    "tax_category_id": "STANDARD_VAT",
    "discount_rate": 0
  }]
}
```

Credit note additions (invoice_type_code = "380"):

Credit notes require a cancel_references block that links the credit note back to the original invoice. The original IRN is read from the TRANS_REF3 column (populated by the ERP when it generates a reversal transaction). Additionally each line item receives a base_quantity field required by the FIRS credit note schema.

```
"cancel_references": [{
  "original_irn": "FT26005XXXXXXXXX-4C5DA4F1-20260105",
  "original_issue_date": "2026-01-05"
}]
```

4. Response Storage — IRN & QR Code

After submitting an invoice payload, the **update_invoice_status()** method processes the Cryptware API response and writes the result back to the REPORT table. Three outcomes are handled.

Outcome 1 — Success (HTTP 201):

Cryptware returns HTTP 201 Created with a JSON body. The agent extracts the IRN and QR code and writes them to the database:

```
# Cryptware success response body:
{
  "status": "success",
  "data": {
    "irn": "FT26008025006184-4C5DA4F1-20260108",
    "qr_code_url": "https://res.cloudinary.com/dxrqdbloy/...",
    "qr_code": "" # fallback if qr_code_url absent
  }
}

# DB update:
UPDATE [dbo].[REPORT]
SET REPORTING_STATUS = 1,
    IRN = "FT26008025006184-4C5DA4F1-20260108",
    QR_CODE = "https://res.cloudinary.com/...",
    ENVIRONMENT = "PREPROD",
    LAST_UPDATED = GETDATE()
WHERE TRANS_REF = "FT26008025006184"
```

Outcome 2 — Validation failure (HTTP 400):

```
# DB update:
UPDATE [dbo].[REPORT]
SET REPORTING_STATUS = 0,
    VALIDATION_ERROR = "Invalid email address | [detail]",
    LAST_UPDATED = GETDATE()
WHERE TRANS_REF = "...

# Record remains eligible for retry on next scheduler cycle.
```

Outcome 3 — Server / network error (HTTP 5xx or timeout):

```
# DB update:
UPDATE [dbo].[REPORT]
SET REPORTING_STATUS = 0,
    VALIDATION_ERROR = "HTTP 500: Unknown error",
    LAST_UPDATED = GETDATE()
WHERE TRANS_REF = "..."
```

REPORTING_STATUS values:

NULL

Pending — never attempted or reset for re-processing

1	Submitted successfully — IRN and QR_CODE are populated
0	Failed — VALIDATION_ERROR contains the reason

QR Code storage:

Cryptware stores QR images on Cloudinary and returns the URL in qr_code_url. The agent stores this URL in the QR_CODE column. The dashboard and PDF generator fetch the image at render time using urllib.request. If a base64 data URI is returned instead (fallback), the dashboard renders it directly as an .

PREPROD archiving: Before switching from the test API key to the production key, run **reset_for_production.sql**. This script copies all PREPROD IRNs to the PREPROD_IRN / PREPROD_QR_CODE backup columns, then resets REPORTING_STATUS to NULL so those invoices are resubmitted to the production API.

5. Compliance Dashboard Server

dashboard.py is a Flask web application that provides a compliance officer interface for monitoring invoice submission status, viewing individual invoice detail, downloading printable PDF invoices with QR codes, and tracking exceptions. It runs on port 5000 and is accessible to all machines on the Keystone Bank internal network.

To start the dashboard:

```
cd e:\EINVOICING_AGENT
.\env\Scripts\python.exe dashboard.py

# Accessible at: http://:5000
```

Routes:

Paragraph ('caseSen sitive': 1 ' encoding' : 'utf8' 'text': 'Method' 'frags': [P araFrag(_ __tag__='p ara', bold=1, fo ntName=' Helvetica- Bold', font Size=8, greek=0, italic=0, link=[], rise=0, tex t='Method' , textColor =Color(.41 9608,,447 059,,5019 61,1), us_l ines=[])] 'style': 'bu lletText': None 'debug': 0) #Paragra ph Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Route' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Route', text Color=Color(.419608,,447059,,50 1961,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph		
/	GET	Dashboard home — stat cards, submissions chart, top errors, recent activity
/invoices	GET	Invoice list — server-side DataTables with search, status filter, date range

<code>/api/invoices</code>	GET	DataTables AJAX endpoint — returns paginated JSON rows with filters applied
<code>/invoices/<ref></code>	GET	Invoice detail — full field display, IRN, QR code image, error detail
<code>/invoices/<ref>/print</code>	GET	Print-optimized HTML — A4 layout with logo, IRN box, QR code. Opens in new tab
<code>/invoices/<ref>/pdf</code>	GET	Downloads a ReportLab-generated PDF — same content as print, for archiving
<code>/assets/<file></code>	GET	Static assets (logo.png, etc.) served from the assets/ directory

Dashboard features:

- **Stat cards** — live counts for Total, Submitted, Pending, and Exceptions
- **Submissions chart** — Chart.js bar chart of daily submission counts for the last 14 days
- **Top exception messages** — ranked list of the most frequent API error messages
- **Invoice search** — full-text search across TRANS_REF, customer name, and IRN
- **Status filter** — quick-filter pills (All / Submitted / Pending / Exceptions)
- **Date range filter** — filter by booking date range
- **PREPROD label** — amber warning badge in the topbar reminds staff the system is on PREPROD
- **Print invoice** — FIRS-compliant A4 print layout with bank logo, IRN box, customer details, line items, VAT calculation, QR code image, and generation timestamp
- **PDF download** — ReportLab-generated PDF identical in content to the print view

PDF generation (ReportLab):

The `generate_pdf()` function in `dashboard.py` uses ReportLab Platypus to build a structured A4 PDF. The QR code image is fetched from the Cloudinary URL at render time using `urllib.request` and embedded as an inline image. The PDF includes:

- Keystone Bank logo (`assets/logo.png`) in the header
- IRN in a highlighted blue banner
- Invoice metadata table (number, date, currency, type, IRN)
- QR code image (42mm × 42mm) beside the metadata
- Bill-To section with customer name, TIN, account, address, phone
- Invoice lines table with description, HSN code, tax category, rate, amount
- Subtotal / VAT / Total summary rows
- FIRS compliance footer with generation timestamp

6. Automated Invoice Submission Service

The automated service is implemented in `invoice_scheduler_pymssql.py` using **APScheduler** (BlockingScheduler). It runs as a persistent background process on the Keystone Bank server, polling the database every 5 minutes and submitting any pending or failed invoices to the Cryptware NRS portal.

To start the scheduler:

```
cd e:\EINVOICING_AGENT
.\env\Scripts\python.exe invoice_scheduler_pymssql.py

# Or run once (no loop):
.\env\Scripts\python.exe run_once.py
```

Processing cycle (every 5 minutes):

Step 1 — Query pending records

`SELECT * FROM [dbo].[REPORT] WHERE REPORTING_STATUS IS NULL OR REPORTING_STATUS = 0`
Fetches all rows that have never been submitted (NULL) or previously failed (0). Failed records are retried on every cycle until they succeed.

Step 2 — Group by TRANS_REF

Multiple rows can share the same TRANS_REF if an invoice has more than one line item. The scheduler groups them so all lines are sent in a single API call.

Step 3 — Build payload

`map_to_api_payload()` applies all normalisation helpers and constructs the Cryptware JSON payload, including the correct `invoice_type_code`, B2B/B2C classification, and any `cancel_references` for credit notes.

Step 4 — POST to API

HTTP POST to `https://preprod-api.cryptwaresystems ltd.com/invoice/generate` with X-API-Key header. Timeout: 30 seconds.

Step 5 — Parse response

HTTP 201 → extract `irn` and `qr_code_url` from `response.data`.
HTTP 400 → extract validation error message.
HTTP 5xx → record generic error. All outcomes update the DB immediately.

Step 6 — Update database

`update_invoice_status()` writes `REPORTING_STATUS`, `IRN`, `QR_CODE`, `ENVIRONMENT`, `VALIDATION_ERROR`, and `LAST_UPDATED` for the processed TRANS_REF.

Step 7 — Sleep and repeat

APScheduler waits 5 minutes (configurable via scheduler.interval_minutes in config.json) then runs the cycle again. Only truly pending/failed records are picked up — successfully submitted invoices (REPORTING_STATUS = 1) are never re-submitted.

Logging:

All scheduler activity is written to **invoice_scheduler.log** (rotating, appended each run) and also echoed to stdout. Each line is timestamped. A typical successful run looks like:

```
2026-01-08 15:53:01,412 INFO Found 12 pending invoice records
2026-01-08 15:53:01,413 INFO Grouped into 12 unique invoices
2026-01-08 15:53:01,450 INFO Processing invoice: FT26008025006184
2026-01-08 15:53:01,451 INFO Submitting invoice FT26008025006184 to API
2026-01-08 15:53:02,103 INFO API response for FT26008025006184: 201
2026-01-08 15:53:02,145 INFO Successfully updated FT26008025006184 with IRN: FT260...
```

File structure summary:

Paragraph('caseSensitive': 1 'encoding': 'utf8'
'text': 'File' 'frags': [ParaFrag(__tag__='para',
bold=1, fontName='Helvetica-Bold', fontSize=8,
greek=0, italic=0, link=[], rise=0, text='File',
textColor=Color(.419608,.447059,.501961,1),
us_lines=[])] 'style': 'bulletText': None 'debug':
0) #Paragraph

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Purpose' 'frags':
[ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8,
greek=0, italic=0, link=[], rise=0, text='Purpose',
textColor=Color(.419608,.447059,.501961,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0) #Paragraph

config.json	DB credentials, API key, scheduler interval (gitignored)
invoice_scheduler_pymssql.py	Main scheduler — InvoiceScheduler class + APScheduler entry point
invoice_scheduler.py	Alternate version (identical logic, legacy reference)
run_once.py	Run one processing cycle without starting the APScheduler loop
dashboard.py	Flask web server — all routes, PDF generator, DataTables API
templates/base.html	Bootstrap 5 sidebar layout, Keystone blue theme, PREPROD badge
templates/index.html	Dashboard home — stat cards, chart, recent activity
templates/invoices.html	Invoice list with DataTables, search, and filter controls
templates/invoice_detail.html	Invoice detail page — full field display, IRN, QR, error
templates/invoice_print.html	Print-optimized standalone A4 HTML invoice
assets/logo.png	Keystone Bank logo used in print view and PDF
setup_database.sql	ALTER TABLE script that adds compliance columns to REPORT
reset_for_production.sql	Resets preprod records (archives IRN/QR) before production run
.gitignore	Excludes config.json, venv/, *.log, *.zip from git